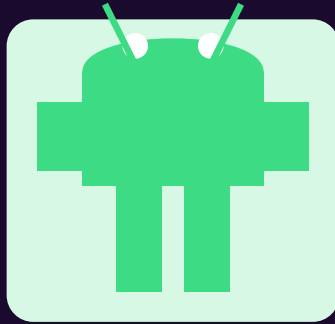




Android Versions App

Documentacion Tecnica — Parcial 2

Autor: Diego Alejandro Groot Cabrera

Profesor: Adolfo Centeno

Entrega: 8 de Mayo de 2026

Flutter | Spring Boot | PostgreSQL | Docker | Render

0 Índice

1. Descripcion General del Proyecto	2
2. Tecnologias Utilizadas	2
3. Patrones de Diseno Utilizados	2
3.1. Patron Facade	3
3.1.1. Por que se uso en este proyecto?	3
3.1.2.Codigo implementado	3
3.2. Patron Repository	5
3.2.1. Por que se uso en este proyecto?	5
3.2.2. Codigo implementado	5
3.3. Patron MVC (Model – View – Controller)	5
3.3.1. Como se aplico en el proyecto?	6
3.3.2. Codigo implementado	6
4. Endpoints de la API REST	7
5. Arquitectura y Despliegue	7
6. Conclusion	8

1 Descripción General del Proyecto

Resumen: Aplicación multiplataforma para registrar versiones del sistema operativo Android, con soporte de reacciones emoji y comentarios por publicación. Backend REST con Spring Boot y frontend Flutter.

La idea del proyecto es tener una especie de red social enfocada en versiones de Android. Cada usuario puede crear una publicación con el nombre de la versión, su fecha, descripción y características. Otros usuarios pueden reaccionar con emojis (like, love, haha, wow, sad) y también dejar comentarios de texto.

El backend se hizo con **Spring Boot** y expone una API REST. La base de datos es **PostgreSQL**, tanto local como en la nube con Aiven (producción). El frontend se construyó con **Flutter**, compilando la misma base de código para web y APK de Android. El proyecto está desplegado en **Render** usando **Docker**.

2 Tecnologías Utilizadas

Tecnología	Uso en el proyecto
Flutter (Dart)	Interfaz: web, Linux desktop y APK Android
Spring Boot 4	Framework backend, expone la API REST
PostgreSQL 16	Base de datos relacional en producción y local
Hibernate / JPA	Mapeo objeto-relacional de las entidades
Docker	Contenedorización del backend para despliegue
Render	Plataforma de despliegue en la nube
Maven	Gestión de dependencias y build del proyecto Java
GitHub	Control de versiones

Cuadro 1: Stack tecnológico del proyecto

3 Patrones de Diseño Utilizados

En el proyecto se utilizaron tres patrones de diseño de software: **Facade**, **Repository** y **MVC**. Cada uno resuelve un problema diferente en la arquitectura de la aplicación.

3.1 Patron Facade

Que es el patron Facade?

La idea del Facade es poner una capa intermedia que oculte la complejidad de un sistema. En lugar de que quien llama tenga que saber con cuantas clases interactuar, hay una sola clase fachada que lo resuelve todo internamente. Es como un cajero automatico: uno solo presiona los botones y la maquina se encarga del resto sin que uno sepa que pasa adentro.

Por que se uso en este proyecto?

Los controladores REST necesitaban hacer varias operaciones con la base de datos: buscar una version de Android por ID, crear un comentario asociado, crear una reaccion, contar cuantas reacciones hay por tipo, etc. Sin un Facade, cada controlador tendria que importar directamente varios repositorios. Con **TweetFacade** se centraliza toda esa logica y los controladores solo llaman a la fachada con lo minimo necesario.

Codigo implementado

TweetFacade.java

```
1  @Service
2  public class TweetFacade {
3
4      private final AndroidRepository tweetRepository;
5      private final ComentarioRepository comentarioRepository;
6      private final ReaccionRepository reaccionRepository;
7
8      public Comentario agregarComentario(Long tweetId, String
9      texto) {
10         android_tweet tweet = tweetRepository.findById(tweetId)
11             .orElseThrow(() -> new RuntimeException(
12                 "El Tweet con ID " + tweetId + " no existe."));
13         return comentarioRepository.save(new Comentario(texto,
14             tweet));
15     }
16
17     public Reaccion agregarReaccion(Long tweetId, String tipo) {
18         android_tweet tweet = tweetRepository.findById(tweetId)
19             .orElseThrow(() -> new RuntimeException("Tweet no
20             encontrado"));
21         return reaccionRepository.save(new Reaccion(tipo, tweet))
22     };
23
24     public Map<String, Long> contarReacciones(Long tweetId) {
25         List<Reaccion> reacciones =
26             reaccionRepository.findByTweetId(tweetId);
27         Map<String, Long> conteo = new HashMap<>();
28         for (Reaccion r : reacciones) {
```

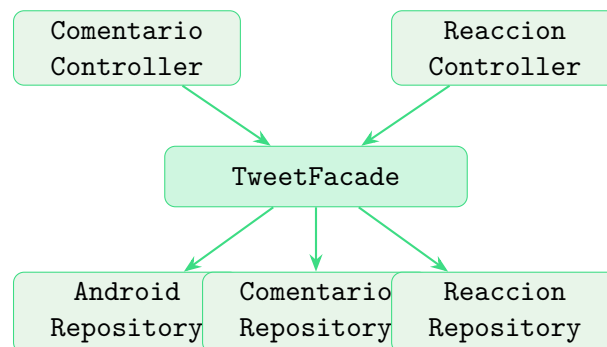
```
26         conteo.merge(r.getTipo(), 1L, Long::sum);
27     }
28     return conteo;
29 }
30 }
```

El controlador usa la fachada sin saber nada de como funciona internamente:

ComentarioController.java

```
1  @CrossOrigin(origins = "*")
2  @RestController
3  @RequestMapping("/api/tweets")
4  public class ComentarioController {
5
6      private final TweetFacade tweetFacade;
7
8      @PostMapping("/{id}/comentarios")
9      public ResponseEntity<?> agregar(
10          @PathVariable Long id,
11          @RequestBody Map<String, String> body) {
12          try {
13              Comentario c = tweetFacade.agregarComentario(
14                  id, body.get("texto"));
15              return ResponseEntity.ok(c);
16          } catch (RuntimeException e) {
17              return ResponseEntity.status(404)
18                  .body(Map.of("error", e.getMessage()));
19          }
20      }
21  }
```

Diagrama de como la fachada separa los controladores de los repositorios:



3.2 Patron Repository

Que es el patron Repository?

El Repository actua como una coleccion de objetos en memoria. En lugar de escribir SQL directamente en la logica de negocio, se crea una interfaz que define que operaciones se pueden hacer con cada entidad. Spring Data JPA genera automaticamente la implementacion a partir del nombre de los metodos, sin necesidad de escribir SQL.

Por que se uso en este proyecto?

Separar el acceso a datos de la logica hace que el codigo sea mas limpio. Si se cambia la base de datos, solo hay que modificar el repositorio y no toda la aplicacion. Spring Data JPA genera el SQL automaticamente a partir del nombre del metodo, lo que reduce errores y codigo repetitivo.

Codigo implementado

ComentarioRepository.java

```
1 @Repository
2 public interface ComentarioRepository
3     extends JpaRepository<Comentario, Long> {
4
5     // Spring genera automaticamente:
6     // SELECT * FROM comentarios WHERE tweet_id = ?
7     List<Comentario> findByTweetId(Long tweetId);
8 }
```

ReaccionRepository.java

```
1 @Repository
2 public interface ReaccionRepository
3     extends JpaRepository<Reaccion, Long> {
4
5     List<Reaccion> findByTweetId(Long tweetId);
6     long countByTweetId(Long tweetId);
7 }
```

3.3 Patron MVC (Model – View – Controller)

Que es el patron MVC?

MVC divide la aplicacion en tres capas: el **Modelo** maneja los datos, la **Vista** muestra la interfaz al usuario, y el **Controlador** recibe las peticiones y coordina entre los otros dos. Esto permite cambiar la interfaz sin tocar la logica del negocio.

Como se aplica en el proyecto?

MVC se aplica en dos niveles. En el backend, Spring Boot usa `@RestController` como controladores y las clases `@Entity` como modelos. En el frontend, Flutter actúa como la vista que consume la API y muestra los datos al usuario.

Código implementado

Modelo — entidad JPA que mapea la tabla de reacciones:

Reaccion.java -- Modelo

```
1 @Entity
2 @Table(name = "reacciones")
3 public class Reaccion {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String tipo;
10
11     @ManyToOne(fetch = FetchType.LAZY)
12     @JoinColumn(name = "tweet_id")
13     private android_tweet tweet;
14 }
```

Vista — pantalla de detalle en Flutter:

detail_screen.dart -- Vista (Flutter)

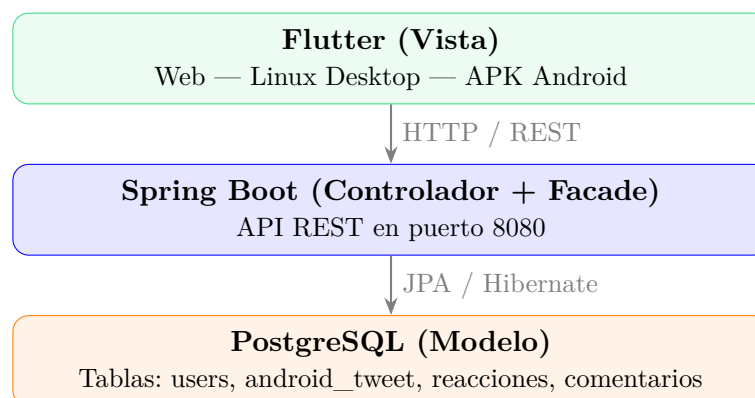
```
1 // La vista solo llama al servicio y actualiza la UI
2 Future<void> _reaccionar(String tipo) async {
3     await AndroidService.agregarReaccion(
4         widget.version.id, tipo);
5     await _cargarReacciones();
6 }
7
8 Future<void> _enviarComentario() async {
9     final texto = comentarioCtrl.text.trim();
10    await AndroidService.agregarComentario(
11        widget.version.id, texto);
12    comentarioCtrl.clear();
13    await _cargarComentarios();
14 }
```

4 Endpoints de la API REST

Metodo	Ruta	Descripcion
Versiones de Android		
GET	/api/android-versions	Obtener versiones de un usuario
POST	/api/android-versions	Crear nueva version
DELETE	/api/android-versions/{id}	Eliminar version por ID
Usuarios		
POST	/api/users	Registrar nuevo usuario
POST	/api/users/login	Iniciar sesion
PUT	/api/users/{id}	Actualizar datos del usuario
DELETE	/api/users/{id}	Eliminar cuenta
Reacciones y Comentarios		
POST	/api/tweets/{id}/reacciones	Agregar reaccion
GET	/api/tweets/{id}/reacciones/conteo	Conteo por tipo
POST	/api/tweets/{id}/comentarios	Agregar comentario
GET	/api/tweets/{id}/comentarios	Lista de comentarios

Cuadro 2: Endpoints completos de la API REST

5 Arquitectura y Despliegue



El backend se empaqueta con Maven en un JAR y se contenedoriza con Docker. La imagen se sube a Docker Hub y Render la descarga para el despliegue. Las variables de entorno DATABASE_URL, DB_USER y DB_PASSWORD configuran la conexión a PostgreSQL en Aiven.


```
$ mvn spring-boot:run -Dspring-boot.run.profiles=local  
$ flutter run -d chrome
```

6 Conclusion

El uso de los patrones **Facade**, **Repository** y **MVC** permitio organizar el proyecto de una forma clara donde cada clase tiene una sola responsabilidad.

La **fachada** fue el patron mas importante porque evito que los controladores tuvieran que conocer los detalles de como se guarda un comentario o como se cuentan las reacciones. Todo queda encapsulado en **TweetFacade** y los controladores simplemente delegan.

Los **repositorios** hicieron que el acceso a datos fuera muy limpio. Spring Data JPA genera el SQL automaticamente, lo que reduce errores y codigo repetitivo.

El patron **MVC** permitio desarrollar el backend y el frontend de forma independiente. Flutter no sabe nada de la base de datos: solo consume la API REST y muestra los datos al usuario.